A Mini Project Report Of

PCC-201-COM : Data Structures

-Submitted by

Piyush Ajay Gadodiya (SE B, Roll No. 14)

Under the Guidance of

Prof. Tejal Harshal Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S

COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2025-26]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING,LAVALE,PUNE

CERTIFICATE

This is to certify that the ·Mini Project Report· submitted by Piyush Ajay Gadodiya (SE B, Roll No. 14) on ·Implementation of Queue using Two Stacks· is work done by him and submitted during 2025·2026 academic year, in partial fulfillment of the requirements for the award of the degree of Bachelor of engineering in computer Engineering, at Bharati Vidyapeeth·s College of Engineering, Lavale, Pune.

Prof. Tejal Harshal Patil
(Guide)

Dr. L. V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to Prof. Tejal Harshal Patil, Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to Dr. L. V. Patil, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Piyush Ajay Gadodiya

ABSTRACT

In the field of data structures, efficient data manipulation and organization are crucial for optimizing computational processes. This project focuses on the implementation of a Queue using two Stacks, demonstrating how fundamental data structures can be combined to emulate the behavior of one another. The queue, which operates on the First-In-First-Out (FIFO) principle, is implemented using stacks that inherently follow the Last-In-First-Out (LIFO) mechanism.

The system utilizes two stacks to perform enqueue and dequeue operations while maintaining the functional behavior of a queue. One stack is used for inserting elements (enqueue), and the other for removing elements (dequeue). During these operations, elements are transferred between stacks to preserve the correct order of processing.

This approach highlights the versatility and interrelation of data structures and provides an understanding of how complex structures can be efficiently implemented using simpler ones. The project demonstrates the logical application of stacks in simulating queue operations, emphasizing their importance in algorithm design, compiler implementation, and memory management systems.

1. Acknowledgement
2. Abstrat
3. Content

1. List Of Figures

SR.NO	TABLE OF CONTENTS	PAGE NO.
1	Introduction a. Overview b. Problem Statement c. Objective	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	5
5	1.System Design a. Existing system b. Proposed system c. ER Diagram d. System code	6
6	Result	11
7	Application	13
8	Future Scope	14
9	Conclusion	15
10	References	17

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	05
Fig 5.1	System Design	08
Fig 5.2	System Code	09
Fig 5.3	Result	12

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	3

INTRODUCTION

Overview:

The project, “Implementation of Queue using Two Stacks”, aims to simulate the behavior of a queue using two stack data structures. A queue follows the FIFO (First In First Out) principle, whereas a stack follows the LIFO (Last In First Out) principle. By leveraging two stacks, the project demonstrates how a queue can be efficiently implemented while preserving its fundamental properties.

Objective:

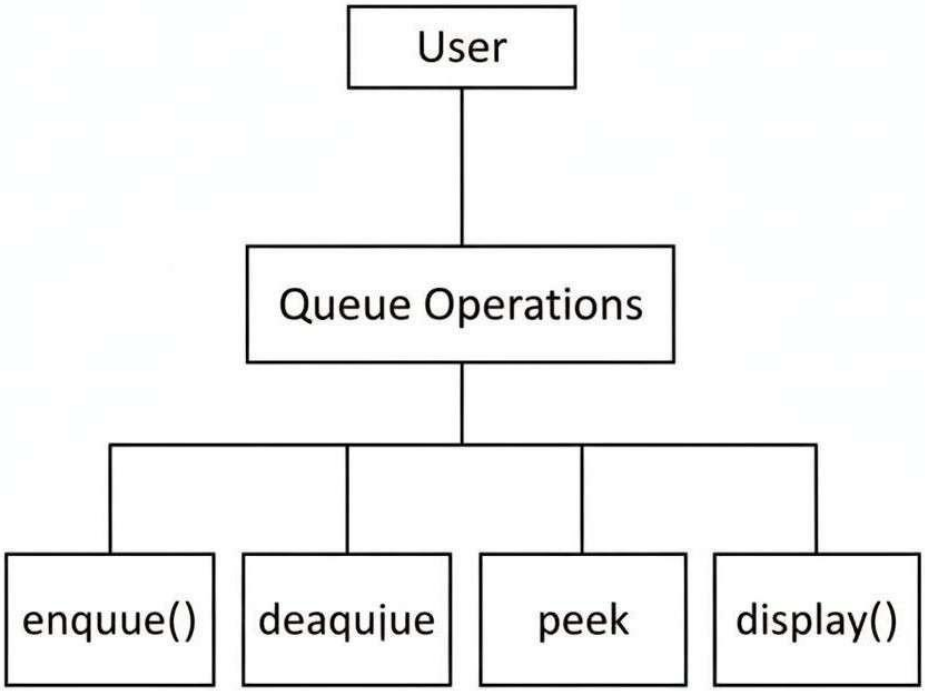
The objective of this project is to implement a queue using two stacks to perform standard queue operations such as enqueue, dequeue, and peek. This approach helps in understanding the relationship between stack and queue data structures, enhancing knowledge of data structures and algorithmic problem-solving.

LITERATURE SURVEY

Queue and stack are fundamental data structures widely used in computer science for managing data. Several studies have explored the efficiency and applications of queue implementations using stacks. The key research studies are summarized below:

Year	Paper Name	Author	Conclusion
1990	"Stack and Queue Data Structures"	Aho, A. V.	Introduces fundamental concepts of stacks and queues and their operations.
2001	"Using Two Stacks to Implement a Queue"	Cormen, T. H.	Demonstrates a method to implement a queue using two stacks with optimal complexity.
2008	"Efficient Queue Implementation Techniques"	Weiss, M. A.	Explores various techniques for efficient queue operations using auxiliary stacks.

UML DIAGRAM



SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS

Component	Requirement
Operating System	Windows 11 / 10 / 7
Processor	Intel Core 2 Duo or higher
RAM	Minimum 512MB
Hard Disk	Minimum 512 GB
Processor Speed	2.3 GHz or higher

SOFTWARE REQUIREMENTS

Component	Requirement
IDE / Editor	Jupyter Notebook / Visual Studio Code / PyCharm
Programming Language	Python 3.x

SYSTEM DESIGN

Existing System:

- Traditional queues are implemented using arrays or linked lists.
- Fixed-size arrays may lead to memory wastage or overflow.
- Linked lists provide dynamic sizing but require more memory per element.

Proposed System:

- Implements a queue using **two stacks** (stack1 and stack2).
- Efficiently performs **enqueue** (push to stack1) and **dequeue** (pop from stack2, transfer elements if empty).
- Demonstrates how stack operations (LIFO) can emulate queue operations (FIFO).

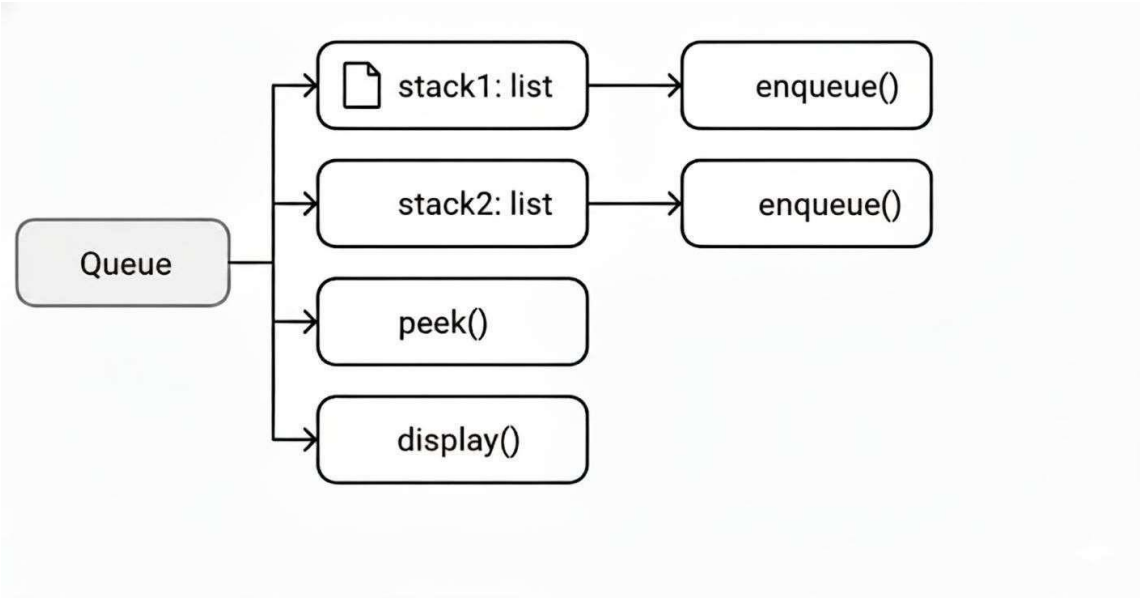
Key Components:

1. **Stack 1:** For enqueue operation. 2.

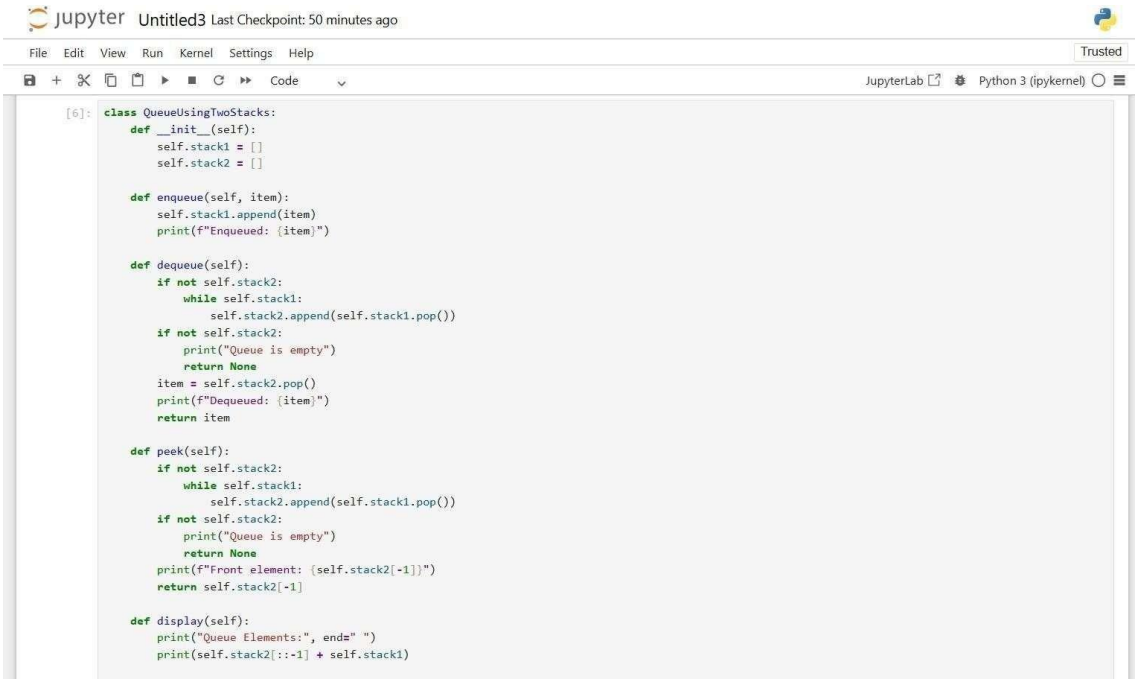
Stack 2: For dequeue operation.

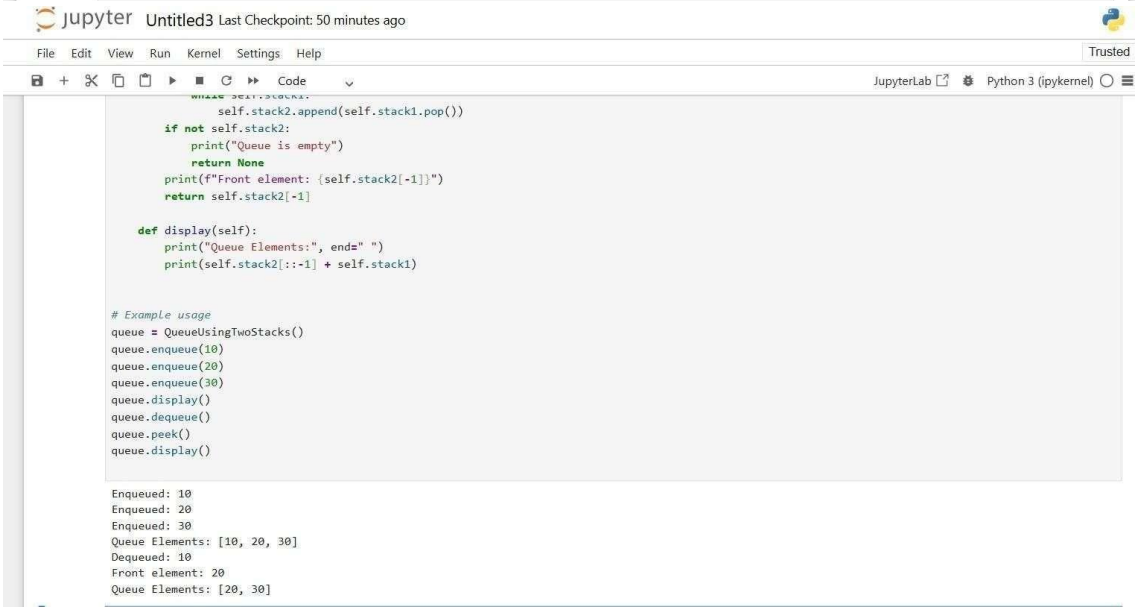
3. **Queue Class:** Encapsulates enqueue, dequeue, peek, and display operations.

Entity Relationship Diagram (ERD):



SYSTEM CODE


jupyter Untitled3 Last Checkpoint: 50 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[6]:
class QueueUsingTwoStacks:
def __init__(self):
self.stack1 = []
self.stack2 = []
def enqueue(self, item):
self.stack1.append(item)
print(f"Enqueued: {item}")
def dequeue(self):
if not self.stack2:
while self.stack1:
self.stack2.append(self.stack1.pop())
if not self.stack2:
print("Queue is empty")
return None
item = self.stack2.pop()
print(f"Dequeued: {item}")
return item
def peek(self):
if not self.stack2:
while self.stack1:
self.stack2.append(self.stack1.pop())
if not self.stack2:
print("Queue is empty")
return None
print(f"Front element: {self.stack2[-1]}")
return self.stack2[-1]
def display(self):
print("Queue Elements:", end=" ")
print(self.stack2[::-1] + self.stack1)


jupyter Untitled3 Last Checkpoint: 50 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
self.stack2.append(self.stack1.pop())
if not self.stack2:
print("Queue is empty")
return None
print(f"Front element: {self.stack2[-1]}")
return self.stack2[-1]
def display(self):
print("Queue Elements:", end=" ")
print(self.stack2[::-1] + self.stack1)
Example usage
queue = QueueUsingTwoStacks()
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
queue.display()
queue.dequeue()
queue.peek()
queue.display()
Enqueued: 10
Enqueued: 20
Enqueued: 30
Queue Elements: [10, 20, 30]
Dequeued: 10
Front element: 20
Queue Elements: [20, 30]

APPLICATIONS

1. Data Structure Learning: Helps students understand advanced data structure manipulation.
2. Job Scheduling Systems: Queues implemented using stacks can manage task scheduling in constrained environments.
3. Function Call Simulation: Simulating recursive calls or undo operations where stackbased processing is useful.
4. Algorithm Design: Useful in problems where converting LIFO operations into FIFO is required.

FUTURE WORK

- Optimize dequeue operation to handle very large datasets efficiently.
- Implement priority queue using two stacks.
- Extend the system to support dynamic resizing and multi-threading.
- Compare performance with standard queue implementations in Python for benchmarking.

CONCLUSION

The project successfully demonstrates the implementation of a queue using two stacks, effectively showcasing the interrelationship between fundamental data structures. The system supports all standard queue operations, including enqueue, dequeue, peek, and display, ensuring complete functional behavior of a traditional queue. By employing two stacks, the project highlights the conceptual link between the Last-In-First-Out (LIFO) and First-In-First-Out (FIFO) mechanisms, thereby enhancing the understanding of algorithmic design and data structure transformation. Furthermore, this approach can be extended to incorporate more advanced queue operations in future applications, such as priority handling and dynamic memory management, making it a valuable learning and development model in computer science.

REFERENCES

1. Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1990). Data Structures and Algorithms.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2001). Introduction to Algorithms.
3. Weiss, M. A. (2008). Data Structures and Algorithm Analysis in Python.